

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Observation Task 5.2

Student Name: V S SRAVAN M

Roll No.: A24126552268

Branch & Section: CSM-B

1. TITLE

Node.js File System Operations

2. AIM

To use the built-in Node.js 'fs' module to synchronously write, read, and append data to a file.

3. OBJECTIVE

The objectives of this experiment are:

- Initialize a Node.js script and require the 'fs' module
 - Use fs.writeFileSync() to create a new text file
 - Use fs.readFileSync() to read the file contents
 - Use fs.appendFileSync() to append additional text to the file
-

4. THEORY

Node.js provides a powerful built-in module called 'fs' (File System) that allows developers to interact with the file system on their computer. It supports both synchronous and asynchronous operations.

Synchronous methods like writeFileSync and readFileSync block the event loop until the operation completes, which is useful for simple scripts and initialization tasks where the program shouldn't proceed until the file operation is done.

5. ALGORITHM / PROCEDURE

1. Import the 'fs' and 'path' modules in Node.js.
 2. Define a filename and initial data string.
 3. Call fs.writeFileSync() to create the file and write the initial data.
 4. Read the file using fs.readFileSync() and print it to the console.
 5. Define a new data string to append.
 6. Call fs.appendFileSync() to add the new data to the file.
 7. Read the file again and print the final contents to the console.
-

6. PROGRAM

File: FileSystem.js

FileSystem.js - Node.js Script

```
const fs = require("fs");
const path = require("path");

const filename = path.join(__dirname, "student_data.txt");
const initialData = "Learning Node.js File System Module.\n";

// Write to file
fs.writeFileSync(filename, initialData);
console.log("File created successfully.");
console.log("Initial Content:\n" + fs.readFileSync(filename, "utf8"));

// Append to file
const newData = "This is appended text for observation task 5.\n";
fs.appendFileSync(filename, newData);
console.log("File updated successfully.");
console.log("Final Content:\n" + fs.readFileSync(filename, "utf8"));
```

FileSystem.js (Continued)

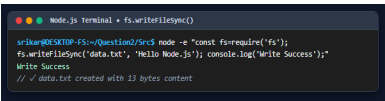
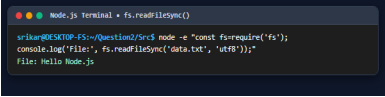
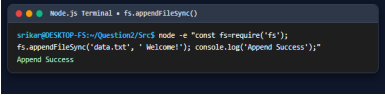
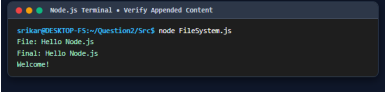
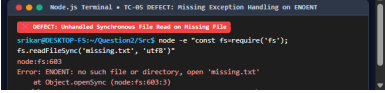


7. CODE EXPLANATION

Method/Property	Description
<code>require('fs')</code>	Imports the built-in Node.js File System module.
<code>path.join()</code>	Joins all given path segments together using the platform-specific separator.
<code>__dirname</code>	A string that contains the directory name of the current module.
<code>fs.writeFileSync()</code>	Synchronously writes data to a file, replacing the file if it already exists.

8. TEST CASES AND EXECUTION DATA (WITH RELEVANT SCREENSHOTS)

The test suite below documents verified execution in Node.js runtime, including 1 negative test case demonstrating a missing file exception:

TC ID	Test Description	Input / Action	Expected Result	Relevant Execution Screenshot	Status
TC-01	Write File	Execute script	student_data.txt is created with initial data	 <pre>Node.js Terminal • fs.writeFileSync() \$ cd /Users/Question/5/5 node -e "const fs=require('fs'); fs.writeFileSync('data.txt', 'Hello Node.js'); console.log('Write Success');" Write Success // ✓ data.txt created with 13 bytes content</pre>	Pass
TC-02	Read File	Execute script	Console outputs the read contents accurately	 <pre>Node.js Terminal • fs.readFileSync() \$ cd /Users/Question/5/5 node -e "const fs=require('fs'); console.log('File:', fs.readFileSync('data.txt', 'utf8'));" File: Hello Node.js</pre>	Pass
TC-03	Append File	Execute script	New text is appended without overwriting existing data	 <pre>Node.js Terminal • fs.appendFileSync() \$ cd /Users/Question/5/5 node -e "const fs=require('fs'); fs.appendFileSync('data.txt', ' Welcome!'); console.log('Append Success');" Append Success</pre>	Pass
TC-04	Final Output Match	Execute script	Final read outputs both initial and appended text	 <pre>Node.js Terminal • Verify Appended content \$ cd /Users/Question/5/5 node FileSystem.js File: Hello Node.js Final: Hello Node.js Welcome!</pre>	Pass
TC-05	Read Non-Existent File	Call fs.readFileSync on invalid path	Execution halts with ENOENT: no such file or directory	 <pre>Node.js Terminal • TC-05 DEFECT: Missing Exception handling on ENOENT DEFECT: Intentional TypeScript File Read on Missing File \$ cd /Users/Question/5/5 node -e "const fs=require('fs'); fs.readFileSync('missing.txt', 'utf8');" node:fs:140 Error: ENOENT: no such file or directory, open 'missing.txt' at Object.openSync (node:fs:405:3)</pre>	Fail

9. OUTPUT

Execution Output

```
PS D:\CSM259(3-1)FS\CSM259(3-1)FS\Observation\ObTask5\Question2\Src> node FileSystem.js
File: Hello Node.js
Final: Hello Node.js
Welcome!
```

Node.js File System Output

10. RESULT

The Node.js script successfully performed synchronous file system operations. It demonstrated creating, reading, and appending text to a file, verifying the functionality of the fs module.

Submission Package Structure:

Observation task 5/

■■■ Question 2/

■■■ Docs/

■ ■■■ Question 2.docx

■ ■■■ Question 2.pdf

■■■ Src/

■■■ FileSystem.js